디자인 패턴

2026. 4. 14 w5
미림 마이스터고

남궁성
seong.namkung@gmail.com

14. Iterator – 처리를 반복. 반복 조건의 분리

14. Iterator – 처리를 반복. 반복 조건의 분리

메서드	설 명
boolean hasNext( )	읽어 올 요소가 남아있는지 확인한다. 있으면 true, 없으면 false를 반환한다.
Object next( )	다음 요소를 읽어 온다. next( )를 호출하기 전에 hasNext( )를 호출해서 읽어 올 요소가 있는지 확인하는 것이 안전하다.
void remove( )	next( )로 읽어 온 요소를 삭제한다. next( )를 호출한 다음에 remove( )를 호출해야한다.(선택적 기능)

```
public interface Iterator {
    boolean hasNext();
    Object next();
    void remove();
        ...
}

public interface Iterable {
    Iterator iterator(); // 모든 Iterable은 Iterator를 제공해야 한다.
        ...
}

public interface Collection extends Iterable { ... }
```

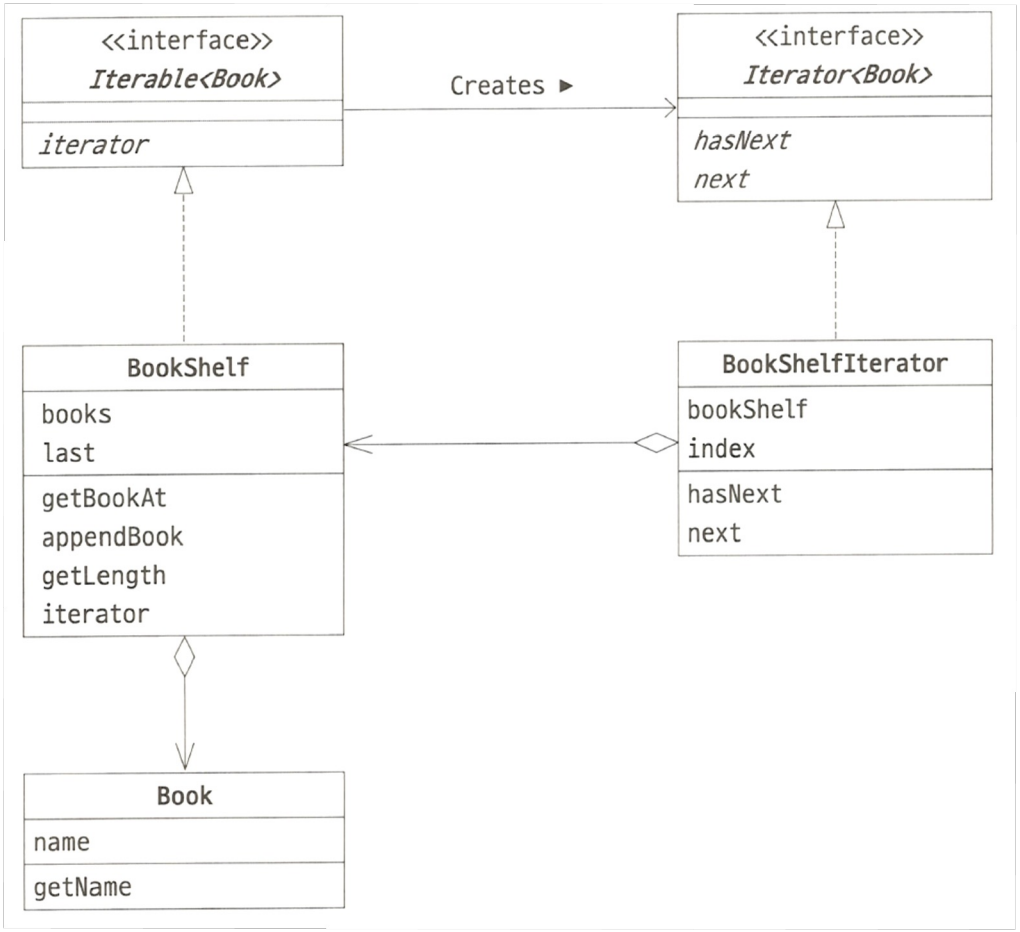
14. Iterator – 처리를 반복. 반복 조건의 분리

```java
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
for (int i = 0; i < list.size(); i++) {
    System.out.println("item = " + list.get(i));
}
```

```java
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
Iterator it = list.iterator();
while(it.hasNext()){
    System.out.println("item = " + it.next());
}
```

```java
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
for(Integer item : list){ // list는 Iterable
    System.out.println("item = " + item);
}
```

14. Iterator – 처리를 반복. 반복 조건의 분리



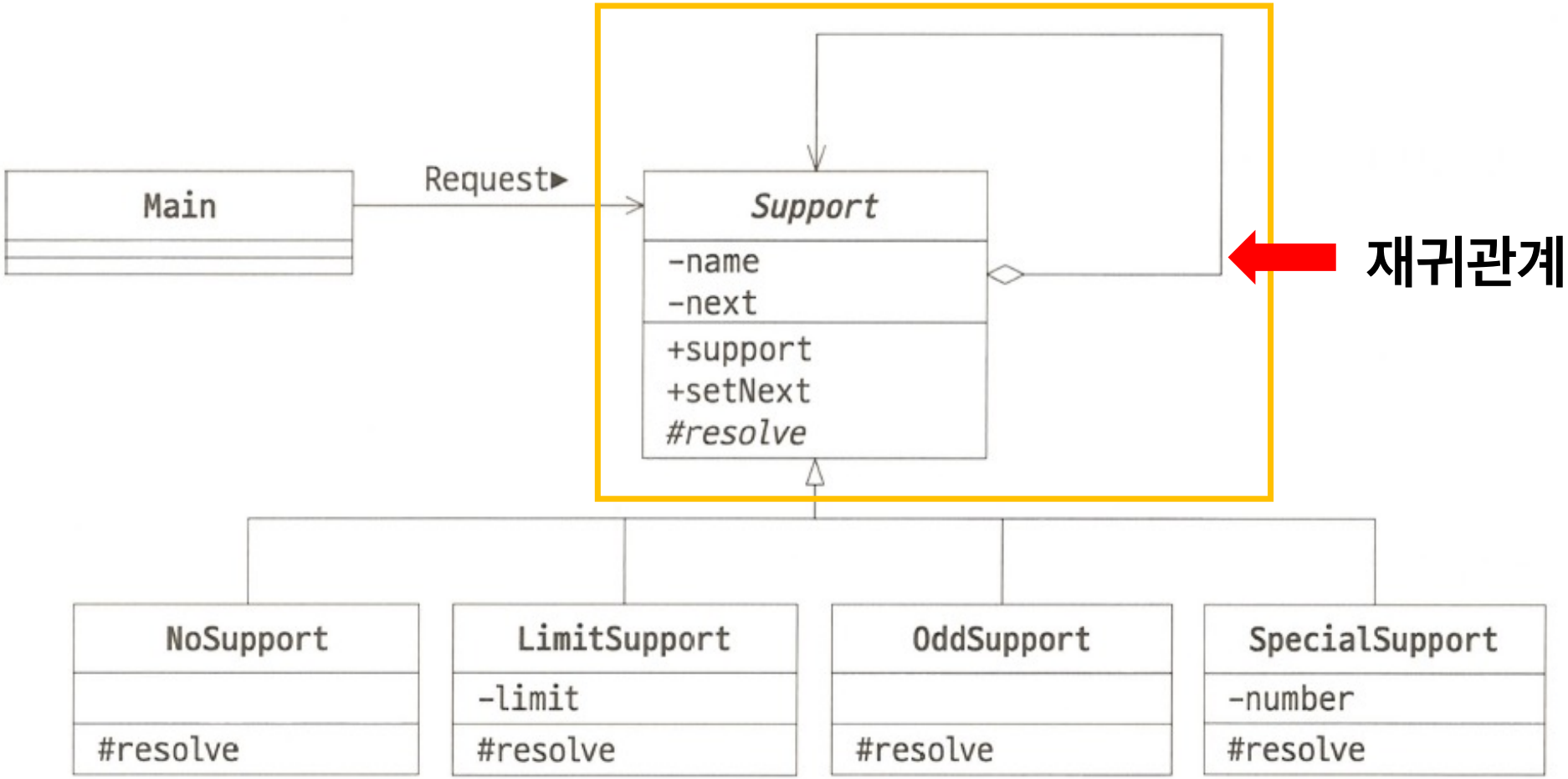
15. Chain of Responsibility – 책임을 떠넘기기

요청이 들어오면, 책임 사슬의 첫 번째 객체가 요청을 처리.
처리할 수 없으면 다음 객체에게 떠넘기기(반복)

1. 여러 객체를 연쇄적으로 묶고(책임사슬),
2. 객체를 순서대로 돌면서 처리할 객체를 결정

요청하는 쪽과 처리하는 쪽의 결합을 약하게 가능. 유연성 증가. 변경에 유리

15. Chain of Responsibility – 책임을 떠넘기기



15. Chain of Responsibility – 책임을 떠넘기기

```
filter1.doFilter()
filter2.doFilter()
servlet.service()
```

→

```
chain.doFilter()
```

하나씩 따로 호출

첫 번째 것만 호출
그 다음 것들은 알아서 호출됨

15. Chain of Responsibility – 책임을 떠넘기기

```
interface Filter {  no usages
    void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)  no usages
            throws IOException, ServletException;
}
```

```
class ApplicationFilterChain implements FilterChain {
    private List<Filter> filters;
    private int index = 0;

    public void doFilter(request, response) {
        if (index < filters.size()) {
            Filter nextFilter = filters.get(index++);
            nextFilter.doFilter(request, response, this);
        } else {
            servlet.service(request, response); // 마지막
        }
    }
}
```

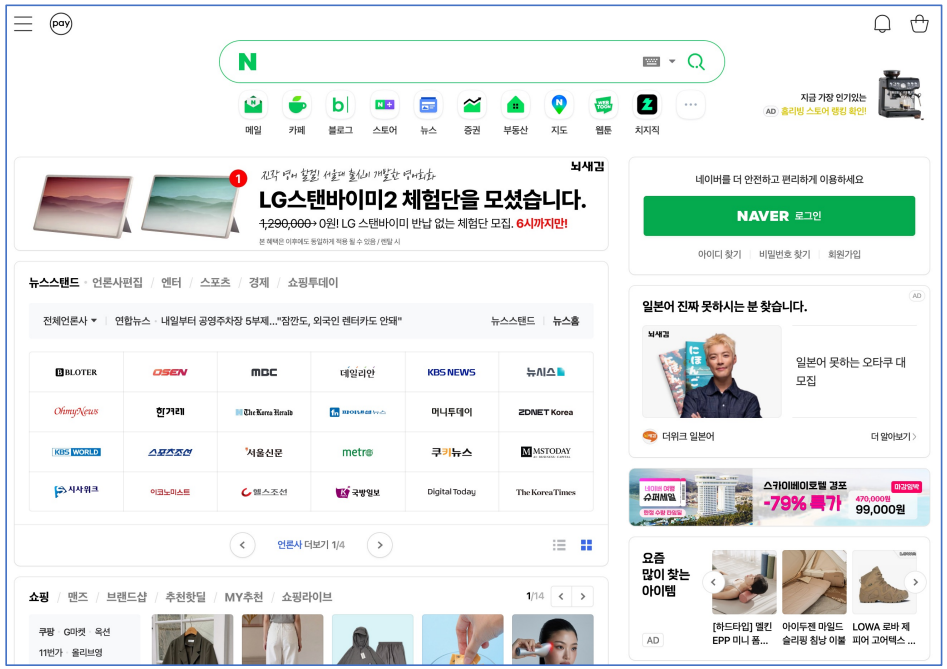
chain

chain에서 순서대로 호출

16. Facade – 단순한 창구 만들기

Facade는 '건물의 정면(face)'이라는 뜻(프랑스어)

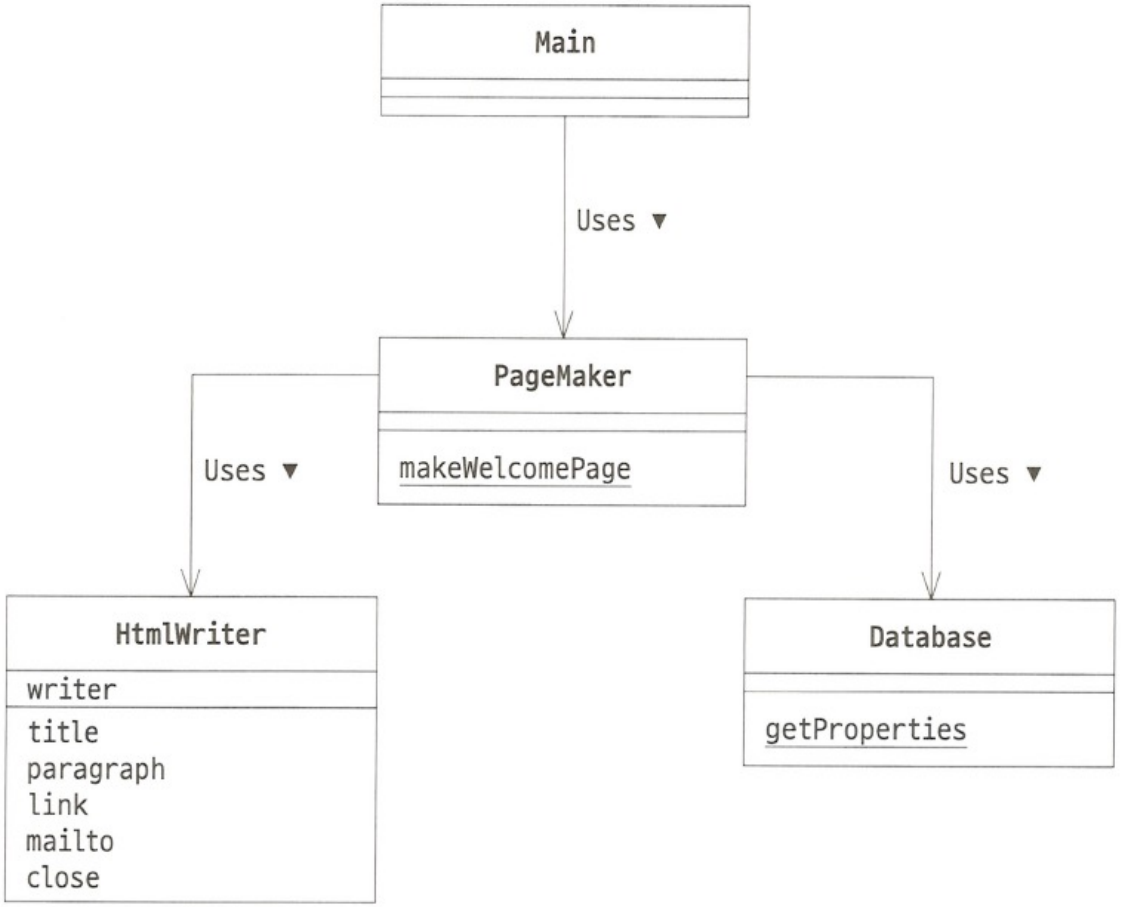
복잡한 세부내용을 정리하여 간단한 인터페이스를 제공



16. Facade – 단순한 창구 만들기

```
public class OrderController {
    public void order() {
        InventoryService inventory = new InventoryService();
        PaymentService payment = new PaymentService();
        ShippingService shipping = new ShippingService();

        inventory.checkStock();
        payment.pay();
        shipping.ship();
    }
}
```

```
public class OrderController {
    private OrderFacade orderFacade = new OrderFacade();

    public void order() {
        orderFacade.placeOrder();
    }
}
```

```
public class OrderFacade {

    private InventoryService inventory = new InventoryService();
    private PaymentService payment = new PaymentService();
    private ShippingService shipping = new ShippingService();

    public void placeOrder() {
        inventory.checkStock();
        payment.pay();
        shipping.ship();
    }
}
```

16. Facade – 단순한 창구 만들기